LAB SHEET 5

Part A Project Setup

1. Create a new folder named express-lab and initialize it with npm init -y.
2. Install express, and install nodemon as a dev dependency.
3. Create index.js with a basic server on port 4000 that responds with “Express Lab Running” on GET /.
4. Add a "dev" script in package.json that runs the server using nodemon.

Solution ->

index.js

express-lab > index.js > ...

```
const express = require("express");
const app = express();
const PORT = 4000;
app.get("/", (req, res) => {
    res.send("Express Lab Running");
});
app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```

Output ->



Part B Basic Routing

5. Create GET /about that returns your name and roll number as a JSON object.
6. Create GET /courses that returns an array of any 3 course names as JSON.
7. Create POST /echo that reads the JSON body sent by the client and returns the exact same data back.

Solution ->

index.js

express-lab > index.js

```
const express = require("express");
const app = express();
const PORT = 4000;
app.get("/", (req, res) => {
    res.send("Express Lab Running");
});
app.get("/about", (req, res) => {
    res.json({
        name: "Vikrant Tyagi",
        rollNo: 67
    });
});
app.get("/courses", (req, res) => {
    res.json([
        "Computer Science",
        "Web Development",
        "Database Management"
    ]);
});
app.post("/echo", (req, res) => {
    res.json(req.body);
});
app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```

Output ->







Part C Route Parameters & Query Strings

8. Create GET /students/:id that returns { id, message: "Student details" } using the id from the URL.

9. Create GET /search that reads query parameters name and age, and returns them in the JSON response.

10. Create GET /products/:category/:id that returns both the category and id captured from the URL.

Solution ->

```
app.get("/students/:id", (req, res) => {
    res.json({
        id: req.params.id,
        message: "Student details"
    });
});
app.get("/search", (req, res) => {
    const { name, age } = req.query;
    res.json({
        name: name,
        age: age
    });
});
app.get("/products/:category/:id", (req, res) => {
    res.json({
        category: req.params.category,
        id: req.params.id
    });
});
app.post("/echo", (req, res) => {
    res.json(req.body);
});
app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```

Output ->







Part D Middleware

11. Write a custom logger middleware that prints the request method, URL, and a timestamp for every incoming request. Apply it globally with app.use().
12. Write a middleware that checks for a custom header x-api-key. If it is missing, respond with 403 Forbidden. Apply this middleware only to routes starting with /admin.
13. Create a protected route GET /admin/dashboard that only returns data when the x-api-key header is present.

Solution ->

```
const logger = (req, res, next) => {
    console.log(
        `${new Date().toISOString()} - ${req.method} ${req.url}`
    );
    next();
};
app.use(logger);
const apiKeyMiddleware = (req, res, next) => {
    const apiKey = req.headers["x-api-key"];
    if (!apiKey) {
        return res.status(403).json({
            error: "Forbidden"
        });
    }
    next();
};
app.use("/admin", apiKeyMiddleware);
app.get("/admin/dashboard", (req, res) => {
    res.json({
        message: "Welcome to Admin Dashboard",
        data: "Protected admin data"
    });
});
```

Part E Working with JSON & Forms

14. Enable express.json() and express.urlencoded({ extended: true }) as middleware.
15. Create a POST /register that accepts { name, email, password } in the request body and returns a success message including the submitted name.
16. Create POST /contact that accepts a urlencoded form submission, logs it to the console, and returns a thank-you message.

Solution ->

```
app.use(express.urlencoded({ extended: true }));
app.post("/register", (req, res) => {
    const { name, email, password } = req.body;
    res.json({
        message: `Registration successful for ${name}`,
        email: email
    });
});
```

Output ->



Part F express.Router() — Modular Routes

17. Create a routes folder. Move all /students routes from Part C into routes/students.js using express.Router().

18. Mount the students router inside index.js under the path /api/students.

19. Repeat the same pattern for a new resource, books: GET /api/books (all books), GET /api/books/:id (one book), POST /api/books (add a book).

Solution ->

```
const express = require("express");
const router = express.Router();
const students = [
    { id: 1, name: "Vikrant", course: "CSE" },
    { id: 2, name: "Rahul", course: "CSE" },
    { id: 3, name: "Aman", course: "CSE" }
];
router.get("/", (req, res) => {
    res.json(students);
});
router.get("/:id", (req, res) => {
    const student = students.find(
        s => s.id == req.params.id
    );
    if (!student) {
        return res.status(404).json({
            error: "Student not found"
        });
    }
    res.json(student);
});
module.exports = router;
```

```
const studentRoutes = require("./routes/students");
app.use("/api/students", studentRoutes);
```

express-lab > routes > JS books.js

```js
const express = require("express");
const router = express.Router();
let books = [
    {
        id: 1,
        title: "The Alchemist",
        author: "Paulo Coelho"
    },
    {
        id: 2,
        title: "Atomic Habits",
        author: "James Clear"
    }
];
router.get("/", (req, res) => {
    res.json(books);
});
router.get("/:id", (req, res) => {
    const book = books.find(
        b => b.id == req.params.id
    );
    if (!book) {
        return res.status(404).json({
            error: "Book not found"
        });
    }
    res.json(book);
});
router.post("/", (req, res) => {
    const newBook = {
        id: books.length + 1,
        title: req.body.title,
        author: req.body.author
    };
    books.push(newBook);
    res.status(201).json(newBook);
});
module.exports = router;
```

```js
const bookRoutes = require("./routes/books");
app.use("/api/books", bookRoutes);
```

Output ->





Part G Building a REST API (CRUD)

20. Choose any resource of your own (e.g. tasks, movies, employees). Using a simple in-memory array, build all 5 CRUD routes for it: GET /api/ · GET /api//:id · POST /api/ · PUT /api//:id · DELETE /api//:id

21. Test all 5 routes using Postman or Thunder Client. Take a screenshot of each successful response for your submission.

Solution ->

express-lab > routes > JS tasks.js

```
const express = require("express");
const router = express.Router();
let tasks = [
    {
        id: 1,
        title: "Learn Express",
        completed: false
    },
    {
        id: 2,
        title: "Practice REST API",
        completed: false
    }
];

router.get("/", (req, res) => {
    res.json(tasks);
});
router.get("/:id", (req, res, next) => {
    const task = tasks.find(
        t => t.id == req.params.id
    );
    if (!task) {
        const error = new Error("Task not found");
        error.status = 404;
        return next(error);
    }
    res.json(task);
});
router.post("/", (req, res) => {
    const newTask = {
        id: tasks.length + 1,
        title: req.body.title,
        completed: req.body.completed || false
    };
    tasks.push(newTask);
    res.status(201).json(newTask);
});

router.put("/:id", (req, res) => {
    const task = tasks.find(
        t => t.id == req.params.id
    );
    if (!task) {
```

```
            return res.status(404).json({
                error: "Task not found"
            });
        }
        task.title = req.body.title ?? task.title;
        task.completed = req.body.completed ?? task.completed;
        res.json(task);
    });
    router.delete("/:id", (req, res) => {
        const index = tasks.findIndex(
            t => t.id == req.params.id
        );
        if (index === -1) {
            return res.status(404).json({
                error: "Task not found"
            });
        }
        const deletedTask = tasks.splice(index, 1);
        res.json({
            message: "Task deleted",
            task: deletedTask[0]
        });
    });
    module.exports = router;
```

Output ->



Part H Error Handling

22. Add a 404 handler (after all routes) that returns { error: "Route not found" } for any unmatched URL.

23. Add a centralized error-handling middleware (4 parameters: err, req, res, next) that logs the error to the console and sends back a JSON error response with the correct status code.

24. In your CRUD API from Part G, make GET /api//:id call next(err) with a 404 error when the given id does not exist.

Solution ->

```
app.use((req, res) => {
    res.status(404).json({
        error: "Route not found"
    });
});
```

```
app.get("/error", (req, res, next) => {
    const error = new Error("Something went wrong");
    error.status = 500;
    next(error);
});
```

```
app.use((err, req, res, next) => {
    console.error(err.message);
    res.status(err.status || 500).json({
        error: err.message || "Internal Server Error"
    });
});
```

Output ->





Part I Static Files & Environment Config

25. Create a public folder containing a simple index.html and style.css. Serve this folder using express.static().

26. Create a .env file storing PORT=4000. Install dotenv and update your server to read the port from process.env.PORT instead of hardcoding it.

Solution ->

```
require("dotenv").config();
const PORT = process.env.PORT || 4000;
const express = require("express");
const app = express();
app.use(express.static("public"));
```

express-lab > public > index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Express Lab</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h1>Express Lab</h1>
    <p>Welcome to my Express.js practical.</p>
</body>
</html>
```

express-lab > public > # Style.css

```
body {
    font-family: Arial, sans-serif;
    text-align: center;
    margin-top: 100px;
}
h1 {
    font-size: 40px;
}
p {
    font-size: 20px;
}
```

Output ->



Express Lab

Welcome to my Express.js practical.